AtlasBridge: Technical Documentation

Architecture, setup and usage for running or integrating AtlasBridge with N-ATLaS

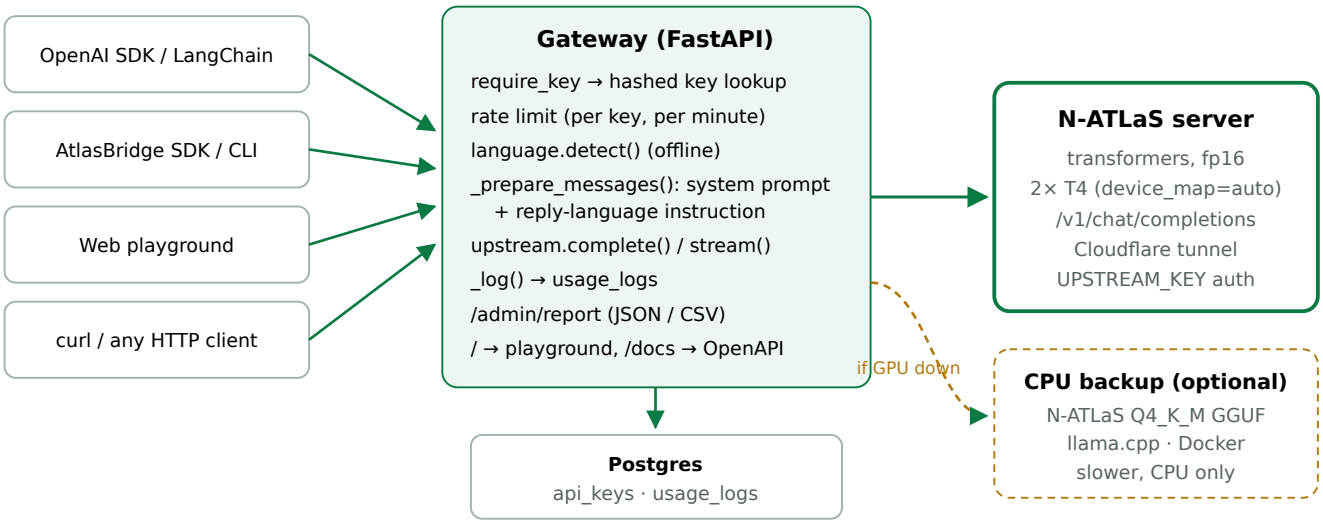
Version	v2.1.0 (10 October 2026)
Repository	https://github.com/Kingsmichaei/atlasbridge
Live gateway	https://atlasbridge.onrender.com (interactive API reference at `/docs`)
Licence	AtlasBridge code: MIT. N-ATLaS model: Awarri / FMCIDE Open-Source Research and Innovation License

1. Overview

N-ATLaS is open, but using it means downloading an 8-billion-parameter model, finding a GPU, and implementing its chat template. AtlasBridge removes those barriers by hosting N-ATLaS behind the API format most developers already use. Any app built on the OpenAI SDK, LangChain or plain HTTP can switch to N-ATLaS by changing its `base_url`, and gets Hausa, Igbo, Yoruba and English.

Component	Path	Role
Model server	`serving/`	Runs `NCAIR1/N-ATLaS` on a GPU as an OpenAI-compatible server. Kaggle notebook plus a standalone script.
CPU backup	`serving/cpu-backup/`	Optional: N-ATLaS Q4_K_M GGUF on llama.cpp (Docker). The gateway fails over to it automatically when configured.
Gateway	`gateway/app/`	FastAPI service: authentication, rate limits, chat and streaming, translation, language ID, usage logging, admin report, playground.
Python SDK + CLI	`sdk/atlasbridge/`	`chat`, `stream`, `translate`, `detect_language`, `Conversation`, and the `atlasbridge` terminal command.
Playground	`gateway/app/static/index.html`	Web page at `/`: beta signup, chat, translate, detect, and copy-paste code.
Deployment	`render.yaml`	Render Blueprint: web service plus a Postgres database.
Tests	`gateway/tests/`	23 tests, including OpenAI SDK compatibility, a real proxy round trip, GPU→CPU failover, and a language-ID evaluation set.

2. Architecture



Request lifecycle (chat). (1) The client sends an OpenAI-format request with `Authorization: Bearer <key>`. (2) The gateway hashes the key (SHA-256), looks it up and applies the per-minute limit. (3) It detects the language of the latest user message. If the client sent no system prompt, it adds the default system prompt and an instruction naming the reply language. (4) It forwards the request to N-ATLaS with model-card defaults: to the GPU server if it is healthy, otherwise to the CPU backup if one is configured (health is cached for 20 s, and a backend that fails before replying is skipped) (temperature 0.1, repetition penalty 1.12). (5) It streams or returns the reply in OpenAI format, plus `x_atlasbridge.detected_language` and `reply_language`. (6) It writes a usage-log row: key, endpoint, language, latency, tokens, status, client, and optional 300-character previews.

3. Setup

3.1 Prerequisites

- A Hugging Face account that has accepted the access conditions at `huggingface.co/NCAIR1/N-ATLaS`, and a **read** access token.
- A GPU with at least 2× 15 GB or 1× 24 GB VRAM. The free Kaggle "GPU T4 x2" accelerator works (phone-verified account, Internet on).
- Python 3.10+ for the gateway and SDK. A GitHub and Render account for hosting (free tiers work).

3.2 Start the N-ATLaS model server (Kaggle)

1. Kaggle → *New Notebook* → *File* → *Import Notebook* → `serving/atlasbridge_serve_kaggle.ipynb`.
2. *Settings* → *Accelerator*: **GPU T4 x2**. *Internet*: **On**.
3. *Add-ons* → *Secrets*: add `HF_TOKEN` (your Hugging Face token) and `UPSTREAM_KEY` (a long random string the gateway will use).
4. **Run All**. Loading takes 3–15 minutes depending on download speed. The notebook prints `N-ATLaS is up`, then `UPSTREAM_URL = https://….trycloudflare.com`, then test replies in all four languages.

On any other CUDA machine: `pip install torch transformers accelerate fastapi uvicorn`, then `HF_TOKEN=… UPSTREAM_KEY=… python serving/fallback_server.py` (port 8000).

> **Note.** Don't `pip install vllm` on Kaggle. It upgrades PyTorch to a CUDA build that clashes with Kaggle's preinstalled torchaudio ("*PyTorch and TorchAudio were compiled with different CUDA versions*"). The notebook uses the transformers server by default for this reason.

3.3 Run the gateway locally

```
git clone https://github.com/Kingsmichaei/atlasbridge && cd atlasbridge/gateway
pip install -r requirements.txt
cp .env.example .env        # set UPSTREAM_URL, UPSTREAM_KEY, ADMIN_TOKEN
uvicorn app.main:app --reload
# no GPU? set MOCK_UPSTREAM=1 for canned replies while developing
pytest -q                   # 23 tests
```

3.4 Deploy the gateway (Render)

1. Render → *New* + → *Blueprint* → select the repository. `render.yaml` creates the `atlasbridge` web service and the `atlasbridge-db` Postgres database.
2. When prompted, set `UPSTREAM_URL` (from the notebook) and `UPSTREAM_KEY` (the same value as the Kaggle secret). `ADMIN_TOKEN` is generated automatically.
3. Open the service URL. The playground's status dot turns green when N-ATLaS is reachable.
4. When the Kaggle session restarts and gets a new tunnel URL, update `UPSTREAM_URL` under *Environment*. Render redeploys in about 1–2 minutes.

3.5 Optional: CPU backup for failover

For continuous availability, run `serving/cpu-backup/Dockerfile` (llama.cpp serving `tosinamuda/N-ATLaS-GGUF:Q4_K_M`, about 4.9 GB) on any Docker host with about 8 GB of RAM and 2 or more vCPUs. Examples are a VPS, a cloud VM, or a Hugging Face Space on a paid plan, since Docker Spaces now need Hugging Face PRO. Set the `LLAMA_API_KEY` environment variable on the host. Then set `UPSTREAM_FALLBACK_URL` and `UPSTREAM_FALLBACK_KEY` on the gateway. The `keep-awake` GitHub Action pings `GATEWAY_URL` (and `BACKUP_URL` if set) every 10 minutes so free hosts don't sleep. The backup isn't deployed for this submission.

3.6 Configuration

Variable	Default	Purpose
`UPSTREAM_URL`	(empty)	Base URL of the N-ATLaS server.
`UPSTREAM_KEY`	(empty)	Bearer key the model server requires.
`UPSTREAM_FALLBACK_URL`	(empty)	Base URL of the optional CPU backup.
`UPSTREAM_FALLBACK_KEY`	(empty)	Its `LLAMA_API_KEY`.
`FALLBACK_MAX_TOKENS`	`256`	Reply-length cap on the slower backup.
`UPSTREAM_MODEL`	`n-atlas`	Model name sent upstream.
`UPSTREAM_TIMEOUT`	`300`	Seconds to wait for N-ATLaS.
`MOCK_UPSTREAM`	`0`	`1` returns canned replies (development and CI).

`DATABASE_URL`	SQLite file	Postgres in production (`postgres://` is accepted).
`ADMIN_TOKEN`	(empty)	Enables `/admin/*`.
`SIGNUP_CODE`	(empty)	Invite code for self-serve beta keys. Empty disables signup.
`RATE_LIMIT_PER_MINUTE`	`20`	Requests per key per minute.
`LOG_CONTENT`	`0`	`1` stores 300-character prompt and reply previews for quality review.
`CORS_ORIGINS`	`*`	Comma-separated allowed browser origins.

4. Using AtlasBridge

4.1 With the OpenAI SDK (no new library)

```
from openai import OpenAI
client = OpenAI(api_key="ab_…", base_url="https://atlasbridge.onrender.com/v1")
r = client.chat.completions.create(
    model="n-atlas",
    messages=[{"role": "user", "content": "Menene amfanin ruwa ga jikin dan Adam?"}])
print(r.choices[0].message.content)
```

4.2 With the AtlasBridge Python SDK

```
pip install "git+https://github.com/Kingsmichaei/atlasbridge#subdirectory=sdk"

from atlasbridge import AtlasBridge
ab = AtlasBridge(api_key="ab_…", base_url="https://atlasbridge.onrender.com")
ab.chat("Kedu ka ị mere?")                          # replies in Igbo
ab.chat("Tell me about Kano", language="hausa")     # force the reply language
ab.translate("Wash your hands before eating.", to="yoruba")
ab.detect_language("Kini anfaani omi fun ara eniyan?")   # {'language': 'yo', ...}
for piece in ab.stream("Explain mobile money"): print(piece, end="")
chat = ab.conversation(system="You are a farming adviser.")
chat.say("Ina son shuka masara")                    # multi-turn with history
```

CLI: set `ATLASBRIDGE_API_KEY` and `ATLASBRIDGE_BASE_URL`, then run `atlasbridge chat "Sannu"`, `atlasbridge translate --to ig "Good morning"` or `atlasbridge detect "Bawo ni?"`.

4.3 JavaScript and curl

```
const r = await fetch("https://atlasbridge.onrender.com/v1/chat/completions", {
  method: "POST",
  headers: { "Content-Type": "application/json", Authorization: "Bearer ab_…" },
  body: JSON.stringify({ model: "n-atlas", language: "yo",
                         messages: [{ role: "user", content: "Good evening" }] }) });

curl https://atlasbridge.onrender.com/v1/translate -H "Authorization: Bearer ab_…" \
     -H "Content-Type: application/json" -d '{"text":"Good morning","target":"ha"}'
```

5. API reference

Authentication: `Authorization: Bearer <key>`. Auth, rate-limit and upstream errors use OpenAI's shape: `{"error": {"message", "type", "code"}}`, so OpenAI SDKs show them clearly. Status codes: 401 for a missing or invalid key, 429 for the rate limit, 422 for validation errors, 502/503 when N-ATLaS is unreachable.

Method	Path	Request → Response
POST	`/v1/chat/completions`	OpenAI chat body: `messages`, `stream`, `temperature`, `top_p`, `max_tokens` (≤2048), `repetition_penalty`, plus `language` (`auto|en|ha|ig|yo`). Returns an OpenAI `chat.completion` (or SSE chunks) with `x_atlasbridge.detected_language`, `reply_language` and `backend` (`gpu` | `cpu-backup`).

POST	`/v1/translate`	`{text, target, source="auto"}` → `{translation, source, target, model}`
POST	`/v1/detect-language`	`{text, method="heuristic"|"model"}` → `{language, name, confidence, scores, method}`
GET	`/v1/models`	Lists `n-atlas` with its languages and context length (8,192).
GET	`/v1/usage`	The caller's own request counts and average latency per endpoint.
POST	`/v1/signup`	`{name, email, organisation, code}` → `{api_key, …}` (shown once).
GET	`/health`	Gateway version, the state of each N-ATLaS backend, and which one is active. No auth.
POST/GET	`/admin/keys`, `/admin/keys/{id}/revoke`	Create, list and revoke keys (`Bearer ADMIN_TOKEN`).
GET	`/admin/report[?format=csv]`	Validation report: totals, per-language and per-tester usage, success rate, median latency. CSV gives the raw log.

6. Security and privacy

- API keys are random (`ab_` + 192 bits) and stored only as SHA-256 hashes. The plain key is shown once.
- The model server only accepts requests carrying `UPSTREAM_KEY`, so the tunnel URL alone is not usable.
- Admin endpoints compare tokens in constant time and are disabled if `ADMIN_TOKEN` is unset.
- Content logging is off by default. In the beta it is on (`LOG_CONTENT=1`, 300-character previews) to measure quality, and testers are told this in the playground.

7. Testing

`pytest -q` in `gateway/` runs 23 tests: auth and invite codes, chat, streaming, translate, detect, rate limiting, revocation, the admin report, the reply-language instruction, the official **OpenAI Python SDK** against a live local server (normal and streaming), a **real proxy round trip** against a fake OpenAI-compatible upstream (checking that the upstream key and N-ATLaS defaults are forwarded), **GPU→CPU failover** (GPU offline: chat, streaming and translate are served by the backup and logged as `cpu-backup`), the database migration, and the **language-ID evaluation set** (70 phrases, must stay ≥95%; held-out accuracy 29/30).

8. Operations and troubleshooting

Symptom	Cause and fix
Status is amber: "CPU backup, slower"	The GPU server is off, so the backup is answering. For fast replies, restart the Kaggle notebook and update `UPSTREAM_URL` on Render.

Playground shows "Model offline"	The Kaggle session stopped or its tunnel URL changed. Restart the notebook and update `UPSTREAM_URL` on Render (plus the CPU backup, if you run one).
First request takes about 60 s	Only on Render's free plan, which sleeps after 15 minutes idle: open the site once to wake it, or use a paid instance. The live AtlasBridge gateway runs on a paid instance, so it doesn't sleep.
Notebook: 401 / "gated repo"	Accept the N-ATLaS conditions on Hugging Face and check the `HF_TOKEN` secret is attached.
Notebook: CUDA version mismatch	vLLM was installed. Use *Run → Factory reset* and use the default transformers server.
Reply in the wrong language	Pass `language` explicitly, or omit your own system prompt so auto mode can set it.
Kaggle session ends while idle	Keep a cell running that pings `/health` every 5 minutes.

9. Limitations and roadmap

- Answer quality is bounded by N-ATLaS v1. The model card's human evaluation rates Yoruba lowest (2.69/5), against Hausa 3.98, Igbo 3.87 and English 4.21.
- The model runs on free Kaggle GPU sessions, so it isn't always on. The optional CPU backup would keep it available but is slow (about 2–5 tokens/s, 256-token cap) and uses a community 4-bit quantisation, so answers can differ slightly from the fp16 model. Both servers handle one generation at a time. Production would use a persistent GPU host with vLLM for concurrent batching.
- Planned: an N-ATLaS ASR endpoint (`/v1/audio/transcriptions`), JavaScript SDK, per-key quotas, and publishing the SDK to PyPI.
- The licence caps use at 1,000 monthly active end users without an Awarri commercial licence.

N-ATLaS is an initiative of the Federal Ministry of Communications, Innovation and Digital Economy, and powered by Awarri Technologies. AtlasBridge is an independent developer tool built on N-ATLaS.